



Cheat Sheet
Git y GitHub

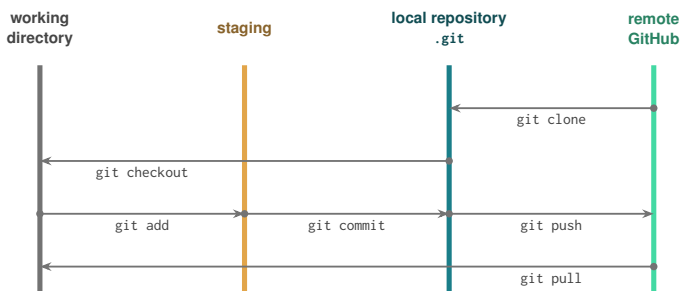
► Crear un repository

<code>git init</code>	Crea un repository nuevo
<code>git clone <url></code>	Clona un repository existente

► Configuración

<code>git config --global user.name "..."</code>	Nombre del autor de cada commit
<code>git config --global user.email "..."</code>	Correo del autor, el mismo de GitHub
<code>git config --global init.defaultBranch main</code>	Define <code>main</code> como branch inicial
<code>git config --list</code>	Muestra la configuración vigente

► Zonas y flujo de trabajo



<code>git add <file></code>	Copia el file al staging
<code>git add .</code>	Añade todo el contenido del working directory
<code>git reset <file></code>	Retira el file del staging
<code>git reset</code>	Vacía por completo el staging
<code>git status</code>	Muestra el estado de cada file

► Hacer commits

<code>git commit</code>	Abre el editor para redactar el mensaje
<code>git commit -m "..."</code>	Crea el commit con el mensaje indicado

► Trabajo con branches

<code>git switch <branch></code> / <code>git checkout <branch></code>	Cambia de branch
<code>git switch -c <branch></code> / <code>git checkout -b <branch></code>	Crea un branch y cambia a él
<code>git branch</code>	Lista los branches existentes
<code>git branch -d <branch></code>	Elimina un branch ya mergeado
<code>git branch -D <branch></code>	Elimina un branch de forma forzada

► Ver diffs

<code>git diff</code>	Working directory frente al staging
<code>git diff --staged</code>	Staging frente al último commit
<code>git diff <c1> <c2></code>	Diff entre dos commits
<code>git diff <commit> <file></code>	Un file desde un commit determinado
<code>git show <commit></code>	Diff de un commit frente a su parent

► Log del proyecto

<code>git log</code>	Registro con hash, autor, fecha y mensaje
<code>git log --oneline</code>	Un commit por línea
<code>git log --graph</code>	Dibuja el grafo de branches y merges
<code>git log --all</code>	Incluye los commits de todos los branches
<code>git log <file></code>	Commits que modificaron un file

► Formas de referirse a un commit

Un branch	<code>main</code>
Un tag	<code>v0.1</code>
Un identificador de commit	<code>3e887ab</code>
Un remote branch	<code>origin/main</code>
El commit actual	<code>HEAD</code>
Tres commits anteriores	<code>HEAD~3</code> o <code>HEAD</code>

► Tags

<code>git tag</code>	Lista los tags existentes
<code>git tag v1.0</code>	Marca el commit actual
<code>git tag -a v1.0 -m "..."</code>	Tag con autor, fecha y mensaje
<code>git tag v1.0 <hash></code>	Marca un commit anterior
<code>git push origin v1.0</code>	Publica un tag concreto
<code>git tag -d v1.0</code>	Elimina el tag local

► Descarte de cambios

<code>git restore <file></code> / <code>git checkout -- <file></code>	Descarta los cambios no añadidos
<code>git restore --staged --worktree <file></code>	Descarta los cambios staged y no staged
<code>git reset --hard</code>	Descarta todo el trabajo sin commitear
<code>git stash</code>	Guarda el trabajo en curso de forma temporal

► Exclusiones con .gitignore

<code>*.csv</code>	Cualquier file con extensión <code>.csv</code>
<code>models/</code>	La carpeta completa
<code>/notas.txt</code>	Únicamente el file de la raíz
<code>**/tmp/</code>	En cualquier nivel del árbol
<code>!data/muestra.csv</code>	Excepción, este file sí se incluye

Si el file ya está commiteado, se retira con `git rm --cached <file>`.

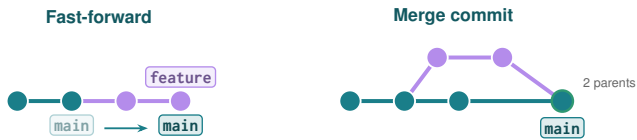
► Reescritura del historial

<code>git commit --amend</code>	Modifica el último commit
<code>git reflog</code>	Recupera commits que quedaron sin branch

► Elección del comando para deshacer

Errata en el último commit aún sin push	<code>git commit --amend</code>
Deshacer commits sin push	<code>git reset <flag> HEAD~1</code>
Deshacer un commit ya publicado	<code>git revert <hash></code>
Descartar los cambios de un file	<code>git restore <file></code>
Recuperar un file de una versión anterior	<code>git checkout <hash> -- <file></code>
Recuperar commits perdidos	<code>git reflog</code>

► Merge de branches



- **Fast-forward** desplaza el branch sin crear un commit.
- **Merge commit** enlaza dos historias con un commit de dos parents.
- **Rebase** reaplica los commits y genera hashes nuevos.
- **Cherry-pick** copia un commit concreto en otro branch.
- **Squash merge** concentra todos los commits en uno.

<code>git merge <branch></code>	Mergea el branch indicado
<code>git merge --no-ff <branch></code>	Fuerza la creación de un merge commit
<code>git rebase <branch></code>	Reaplica los commits sobre el otro branch
<code>git cherry-pick <hash></code>	Copia un único commit
<code>git merge --squash <branch></code>	Prepara el merge en un solo commit

► Conflictos

Se producen cuando dos branches modifican las mismas líneas de un file.

```
<<<<<< HEAD
return m
=====
return round(m, 2)
>>>>>> redondeo
```

La resolución consiste en editar el file, eliminar los marcadores y commitear con `git add` y `git commit`. El merge se cancela con `git merge --abort`.

► Cerrar un branch

<code>git branch --merged</code>	Lista los branches ya mergeados
<code>git branch -d <branch></code>	Elimina el branch y conserva los commits
<code>git branch -D <branch></code>	Elimina el branch de forma forzada

► Remote repository

<code>git remote add <name> <url></code>	Enlaza el repository local con GitHub
<code>git push origin main</code>	Hace push de <code>main</code> al remote <code>origin</code>
<code>git push -u origin <branch></code>	Registra el upstream la primera vez
<code>git push</code>	Hace push al upstream ya registrado
<code>git push --force-with-lease</code>	Reescritura forzada con verificación previa
<code>git push --tags</code>	Hace push de los tags

► Fetch y pull

<code>git fetch origin main</code>	Descarga sin modificar el branch local
<code>git pull</code>	Equivale a <code>fetch</code> y <code>merge</code>
<code>git pull --rebase</code>	Equivale a <code>fetch</code> y <code>rebase</code>

► Fork y pull request

- **Fork** crea una copia del repository en la cuenta propia.
- **Pull request** solicita la incorporación de los cambios.

► Redacción de mensajes de commit

El mensaje es la única parte del commit redactada por el autor y determina la legibilidad del historial.

✗ `cambios · update · asdf · ahora si`
No indica qué cambió ni por qué

✓ `fix: entrena sin el conjunto de prueba · feat: anade el cargador de CSV`
Indica qué cambió y por qué

► Formato Conventional Commits 1.0.0

`<tipo>[<ámbito>]: <descripción>`

tipo clase de cambio **ámbito** parte afectada, opcional **descripción** acción en presente

El **cuerpo** es opcional, se añade con un segundo `-m` y expone el motivo del cambio.

```
git commit -m "fix(modelo): entrena sin el conjunto de prueba" \
-m "Ajustaba con el conjunto de prueba y eso sesgaba la métrica."
```

► Tipos

<code>feat</code>	Funcionalidad nueva	<code>perf</code>	Mejora de rendimiento
<code>fix</code>	Corrección de un error	<code>test</code>	Pruebas
<code>docs</code>	Documentación	<code>ci</code>	Integración continua
<code>style</code>	Formato sin cambio de código	<code>chore</code>	Dependencias y limpieza
<code>refactor</code>	Reestructuración sin cambio funcional		

► Buenas prácticas

- **Modo imperativo** en la descripción, «añade» en lugar de «añadido».
- **Título breve**, de unos 50 caracteres.
- **Un commit por cada cambio lógico**.
- **Referencia al issue** correspondiente.
- **Línea en blanco** entre el título y el cuerpo.

► Ejemplo de un buen commit

```
git commit -m "cambios"
```

```
git commit -m "feat(modelo): anade early stopping" \
-m "El modelo sobreajustaba a partir de la epoca 30. \
Detener por val_loss recorta el entrenamiento a la mitad."
```



Semillero de data science e IA.

Explora lo que hacemos en nuestra web.

aperture-systems-lab.github.io/aperture/links/